Library Database SQL Practice – Questions & Answers

Schema (locked): Publisher(name, address, phone); Book(book_id, title, pub_year, publisher_name); Book_Authors(book_id, author_name); Library_Branch(branch_id, branch_name, address); Book_Copies(book_id, branch_id, no_of_copies); Borrower(card_no, name, address, phone); Book_Loans(book_id, branch_id, card_no, date_out, due_date).

1. Triggers (10)

1. Trigger to update copies after issuing a book

```
CREATE TRIGGER after_issue_update_copies
AFTER INSERT ON Book_Loans
FOR EACH ROW
UPDATE Book_Copies
SET no_of_copies = no_of_copies - 1
WHERE book_id = NEW.book_id AND branch_id = NEW.branch_id;
```

2. Trigger to restore copies after book is returned

```
CREATE TRIGGER after_return_update_copies
AFTER DELETE ON Book_Loans
FOR EACH ROW
UPDATE Book_Copies
SET no_of_copies = no_of_copies + 1
WHERE book_id = OLD.book_id AND branch_id = OLD.branch_id;
```

3. Trigger to prevent issuing if no copies are available

```
CREATE TRIGGER before_issue_check
BEFORE INSERT ON Book_Loans
FOR EACH ROW
BEGIN
IF (SELECT no_of_copies FROM Book_Copies
WHERE book_id = NEW.book_id AND branch_id = NEW.branch_id) <= 0 THEN
SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'No copies available';
END IF;
END;
```

4. Trigger to log whenever a new borrower is added (requires Borrower_Log table)

```
CREATE TABLE IF NOT EXISTS Borrower_Log (
log_id INT AUTO_INCREMENT PRIMARY KEY,
borrower_name VARCHAR(100),
action_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TRIGGER after_borrower_insert
AFTER INSERT ON Borrower
FOR EACH ROW
INSERT INTO Borrower_Log (borrower_name) VALUES (NEW.name);
```

5. Trigger to prevent duplicate book titles

```
CREATE TRIGGER before_insert_book
BEFORE INSERT ON Book
FOR EACH ROW
BEGIN
IF EXISTS (SELECT 1 FROM Book WHERE title = NEW.title) THEN
SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Book already exists';
END IF;
END;
```

6. Trigger to update publisher phone change log (requires Publisher_Log table)

```
CREATE TABLE IF NOT EXISTS Publisher_Log (
publisher_name VARCHAR(100),
old_phone VARCHAR(20),
new_phone VARCHAR(20),
change_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TRIGGER after_publisher_update
AFTER UPDATE ON Publisher
FOR EACH ROW
INSERT INTO Publisher_Log VALUES (OLD.name, OLD.phone, NEW.phone, NOW());
```

7. Trigger to automatically set due_date 15 days after issue

```
CREATE TRIGGER before_book_loan_insert
BEFORE INSERT ON Book_Loans
FOR EACH ROW
SET NEW.due_date = DATE_ADD(NEW.date_out, INTERVAL 15 DAY);
```

8. Trigger to block borrower if they already have 3 issued books

```
CREATE TRIGGER before_loan_limit
BEFORE INSERT ON Book_Loans
FOR EACH ROW
BEGIN
IF (SELECT COUNT(*) FROM Book_Loans WHERE card_no = NEW.card_no) >= 3 THEN
SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Borrower already has 3 books';
END IF;
END;
```

9. Trigger to keep track of deleted borrowers (requires Deleted_Borrowers table)

```
CREATE TABLE IF NOT EXISTS Deleted_Borrowers (
name VARCHAR(100),
address VARCHAR(200),
phone VARCHAR(20),
deleted_on TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TRIGGER after_borrower_delete
AFTER DELETE ON Borrower
FOR EACH ROW
INSERT INTO Deleted_Borrowers (name, address, phone)
VALUES (OLD.name, OLD.address, OLD.phone);
```

10. Trigger to prevent deletion of a publisher if their books exist

```
CREATE TRIGGER before_delete_publisher
BEFORE DELETE ON Publisher
FOR EACH ROW
BEGIN
IF EXISTS (SELECT 1 FROM Book WHERE publisher_name = OLD.name) THEN
SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Cannot delete publisher with
existing books';
END IF;
END;
```

2. Views (10)

1. View showing book titles with their authors

```
CREATE VIEW Book_Author_View AS
SELECT b.title, a.author_name
FROM Book b
JOIN Book_Authors a ON b.book_id = a.book_id;
```

2. View showing branch name and total copies available

```
CREATE VIEW Branch_Copies_View AS
SELECT lb.branch_name, SUM(bc.no_of_copies) AS total_copies
FROM Library_Branch lb
JOIN Book_Copies bc ON lb.branch_id = bc.branch_id
GROUP BY lb.branch_name;
```

3. View of borrowers with their total borrowed books

```
CREATE VIEW Borrower_Loan_Count AS
SELECT br.name, COUNT(bl.book_id) AS total_books
FROM Borrower br
LEFT JOIN Book_Loans bl ON br.card_no = bl.card_no
GROUP BY br.name;
```

4. View to show all overdue books

```
CREATE VIEW Overdue_Books AS
SELECT b.title, br.name AS borrower, bl.due_date
FROM Book_Loans bl
JOIN Book b ON bl.book_id = b.book_id
JOIN Borrower br ON bl.card_no = br.card_no
WHERE bl.due_date < CURDATE();
```

5. View that lists publishers with their number of books

```
CREATE VIEW Publisher_Book_Count AS
SELECT p.name, COUNT(b.book_id) AS total_books
FROM Publisher p
LEFT JOIN Book b ON p.name = b.publisher_name
GROUP BY p.name;
```

6. View showing all books and the branch they belong to

```
CREATE VIEW Branch_Book_List AS
SELECT lb.branch_name, b.title, bc.no_of_copies
FROM Library_Branch lb
JOIN Book_Copies bc ON lb.branch_id = bc.branch_id
JOIN Book b ON bc.book_id = b.book_id;
```

7. View for borrow details report

```
CREATE VIEW Borrow_Details AS
SELECT br.name AS borrower, b.title, bl.date_out, bl.due_date
FROM Borrower br
JOIN Book_Loans bl ON br.card_no = bl.card_no
JOIN Book b ON bl.book_id = b.book_id;
```

8. View for books with more than 10 copies across all branches

```
CREATE VIEW Popular_Books AS
SELECT b.title, SUM(bc.no_of_copies) AS total_copies
FROM Book b
JOIN Book_Copies bc ON b.book_id = bc.book_id
GROUP BY b.title
HAVING total_copies > 10;
```

9. View showing top 3 most issued books

```
CREATE VIEW Top3_Issued_Books AS
SELECT b.title, COUNT(bl.book_id) AS issue_count
```

```
FROM Book_Loans bl
JOIN Book b ON bl.book_id = b.book_id
GROUP BY b.title
ORDER BY issue_count DESC
LIMIT 3;
```

10. View showing books and their publishers' phone numbers

```
CREATE VIEW Book_Publisher_Info AS
SELECT b.title, p.name AS publisher, p.phone
FROM Book b
JOIN Publisher p ON b.publisher_name = p.name;
```

3. Functions (10)

1. Function to count total copies of a book

```
CREATE FUNCTION totalCopies(bookID INT)
RETURNS INT
DETERMINISTIC
RETURN (SELECT IFNULL(SUM(no_of_copies),0) FROM Book_Copies WHERE book_id =
bookID);
```

2. Function to calculate total books borrowed by a borrower

```
CREATE FUNCTION totalBorrowed(card INT)
RETURNS INT
DETERMINISTIC
RETURN (SELECT COUNT(*) FROM Book_Loans WHERE card_no = card);
```

3. Function to get publisher name by book ID

```
CREATE FUNCTION getPublisher(bookID INT)
RETURNS VARCHAR(100)
DETERMINISTIC
RETURN (SELECT publisher_name FROM Book WHERE book_id = bookID);
```

4. Function to find number of books written by an author

```
CREATE FUNCTION booksByAuthor(author VARCHAR(100))
RETURNS INT
DETERMINISTIC
RETURN (SELECT COUNT(*) FROM Book_Authors WHERE author_name = author);
```

5. Function to get borrower name by card number

```
CREATE FUNCTION getBorrowerName(card INT)
RETURNS VARCHAR(100)
DETERMINISTIC
RETURN (SELECT name FROM Borrower WHERE card_no = card);
```

6. Function to calculate fine for overdue books (10 currency units per day)

```
CREATE FUNCTION calculateFine(due DATE)
RETURNS INT
DETERMINISTIC
RETURN GREATEST(0, DATEDIFF(CURDATE(), due)) * 10;
```

7. Function to check if a book is available (returns 1 or 0)

```
CREATE FUNCTION isBookAvailable(bookID INT)
RETURNS BOOLEAN
DETERMINISTIC
RETURN (SELECT IFNULL(SUM(no_of_copies),0) > 0 FROM Book_Copies WHERE book_id =
```

```
bookID);
```

8. Function to get total books in a branch

```
CREATE FUNCTION totalBooksInBranch(branch INT)
RETURNS INT
DETERMINISTIC
RETURN (SELECT IFNULL(SUM(no_of_copies),0) FROM Book_Copies WHERE branch_id =
branch);
```

9. Function to get branch name by ID

```
CREATE FUNCTION getBranchName(branch INT)
RETURNS VARCHAR(100)
DETERMINISTIC
RETURN (SELECT branch_name FROM Library_Branch WHERE branch_id = branch);
```

10. Function to find total publishers

```
CREATE FUNCTION totalPublishers()
RETURNS INT
DETERMINISTIC
RETURN (SELECT COUNT(*) FROM Publisher);
```

4. Stored Procedures (10)

1. Procedure to get all books of a publisher

```
CREATE PROCEDURE getBooksByPublisher(IN pubName VARCHAR(100))
BEGIN
SELECT title FROM Book WHERE publisher_name = pubName;
END;
```

2. Procedure to issue a book (inserts into Book_Loans with 15 day due)

```
CREATE PROCEDURE issueBook(IN bID INT, IN cID INT, IN brID INT)
BEGIN
INSERT INTO Book_Loans(book_id, branch_id, card_no, date_out, due_date)
VALUES(bID, brID, cID, CURDATE(), DATE_ADD(CURDATE(), INTERVAL 15 DAY));
END;
```

3. Procedure to return a book (deletes the loan record)

```
CREATE PROCEDURE returnBook(IN bID INT, IN cID INT)
BEGIN
DELETE FROM Book_Loans WHERE book_id = bID AND card_no = cID;
END;
```

4. Procedure to get borrower details

```
CREATE PROCEDURE getBorrowerDetails(IN cID INT)
BEGIN
SELECT * FROM Borrower WHERE card_no = cID;
END;
```

5. Procedure to add a new branch

```
CREATE PROCEDURE addBranch(IN name VARCHAR(100), IN addr VARCHAR(100))
BEGIN
INSERT INTO Library_Branch(branch_name, address) VALUES(name, addr);
END;
```

6. Procedure to get all books written by an author

```
CREATE PROCEDURE booksByAuthor(IN author VARCHAR(100))
BEGIN
SELECT b.title FROM Book b
```

```
JOIN Book_Authors ba ON b.book_id = ba.book_id
WHERE ba.author_name = author;
END;
```

7. Procedure to update borrower phone number

```
CREATE PROCEDURE updateBorrowerPhone(IN cID INT, IN newPhone VARCHAR(15))
BEGIN
UPDATE Borrower SET phone = newPhone WHERE card_no = cID;
END;
```

8. Procedure to count all overdue books

```
CREATE PROCEDURE countOverdueBooks()
BEGIN
SELECT COUNT(*) AS overdue_count FROM Book_Loans WHERE due_date < CURDATE();
END;
```

9. Procedure to delete a publisher and its books

```
CREATE PROCEDURE deletePublisher(IN pubName VARCHAR(100))
BEGIN
DELETE FROM Book WHERE publisher_name = pubName;
DELETE FROM Publisher WHERE name = pubName;
END;
```

10. Procedure to display total issued books per branch

```
CREATE PROCEDURE branchIssuedBooks()
BEGIN
SELECT lb.branch_name, COUNT(bl.book_id) AS total_issued
FROM Library_Branch lb
JOIN Book_Loans bl ON lb.branch_id = bl.branch_id
GROUP BY lb.branch_name;
END;
```